

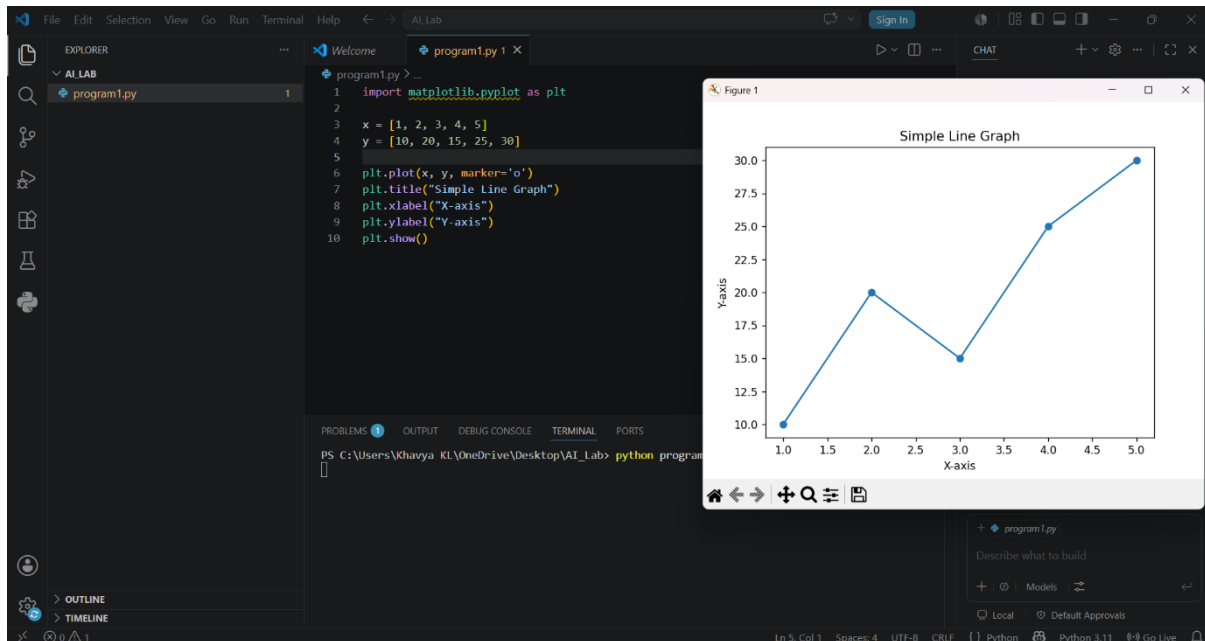
PRACTICE EXPERIMENTS

CSA6501-GENAI

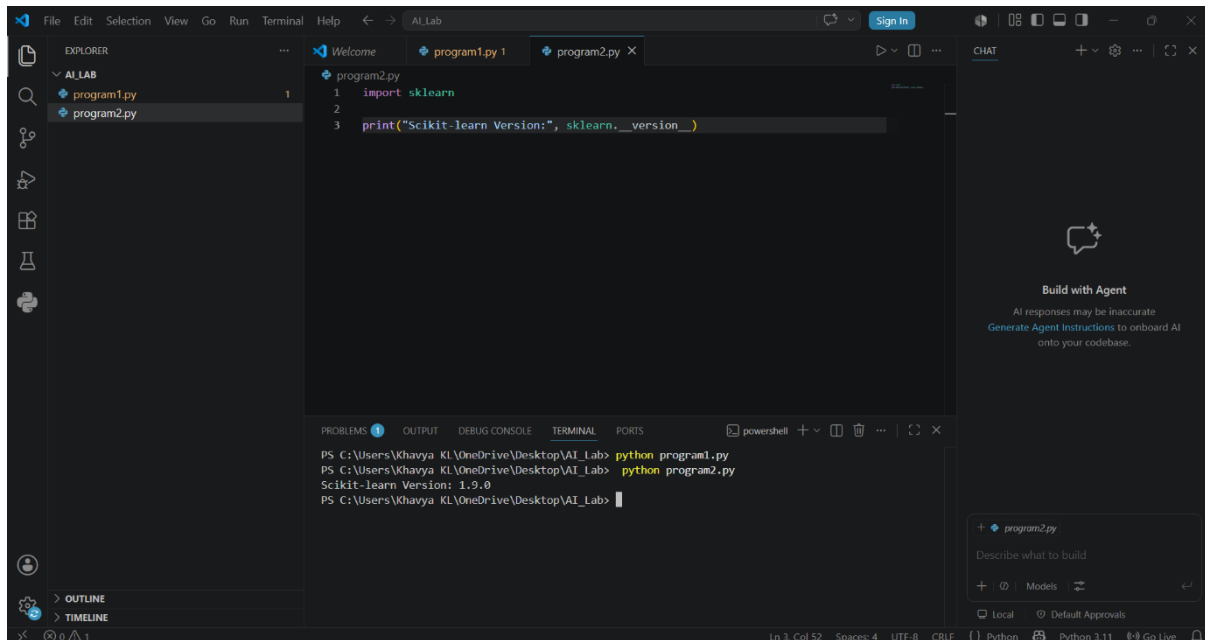
Name: K. L. Khaviya varshini

Reg. No.: 192472381

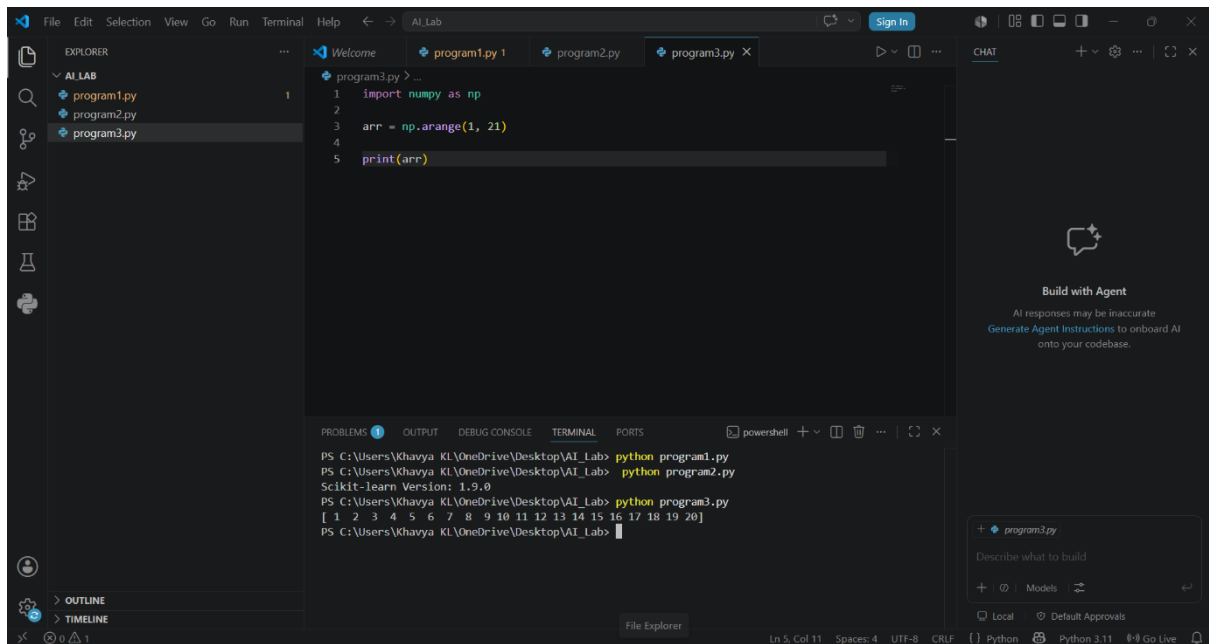
EXP1



EXP2



EXP3



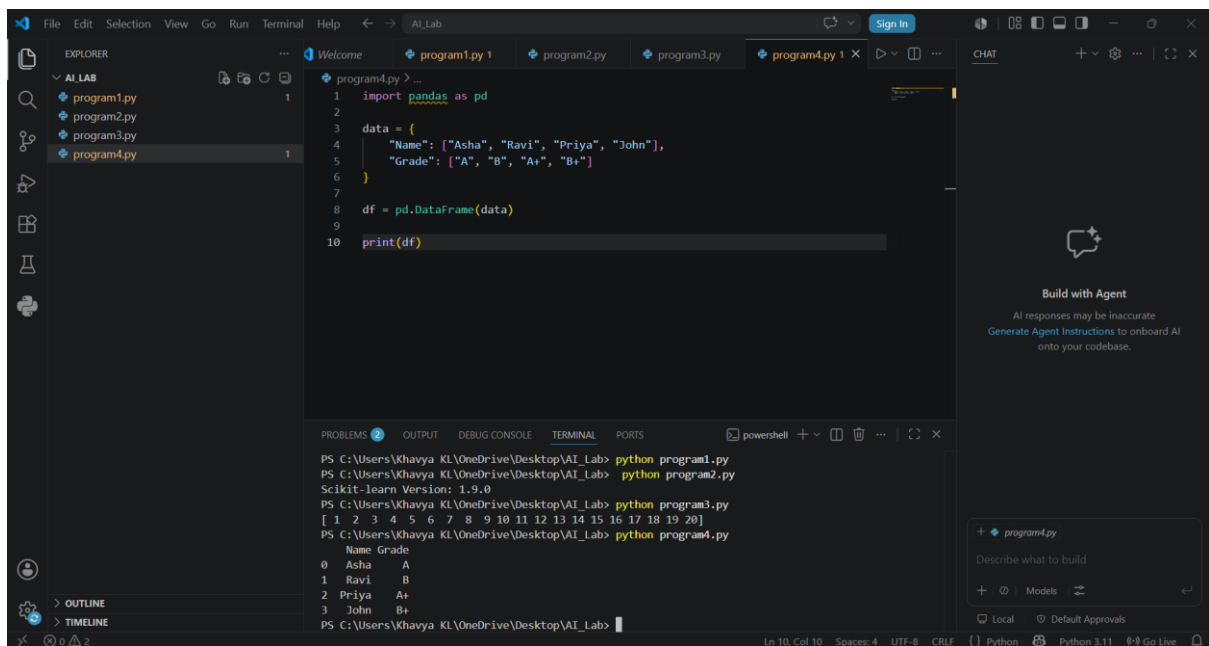
VS Code interface showing the execution of `program3.py`. The code imports `numpy` and prints an array of 21 ones.

```
1 import numpy as np
2
3 arr = np.arange(1, 21)
4
5 print(arr)
```

Terminal output:

```
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program1.py
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program2.py
Scikit-learn Version: 1.9.0
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program3.py
[ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]
```

EXP4



VS Code interface showing the execution of `program4.py`. The code imports `pandas` and prints a DataFrame with columns 'Name' and 'Grade'.

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Asha", "Ravi", "Priya", "John"],
5     "Grade": ["A", "B", "A+", "B+"]
6 }
7
8 df = pd.DataFrame(data)
9
10 print(df)
```

Terminal output:

```
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program1.py
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program2.py
Scikit-learn Version: 1.9.0
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program3.py
[ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program4.py
   Name Grade
0  Asha    A
1  Ravi    B
2  Priya  A+
3  John   B+
```

EXP5

```
1 import numpy
2 import pandas
3 import matplotlib
4 import sklearn
5 import torch
6 import transformers
7
8 print("NumPy:", numpy.__version__)
9 print("Pandas:", pandas.__version__)
10 print("Matplotlib:", matplotlib.__version__)
11 print("Scikit-learn:", sklearn.__version__)
12 print("PyTorch:", torch.__version__)
```

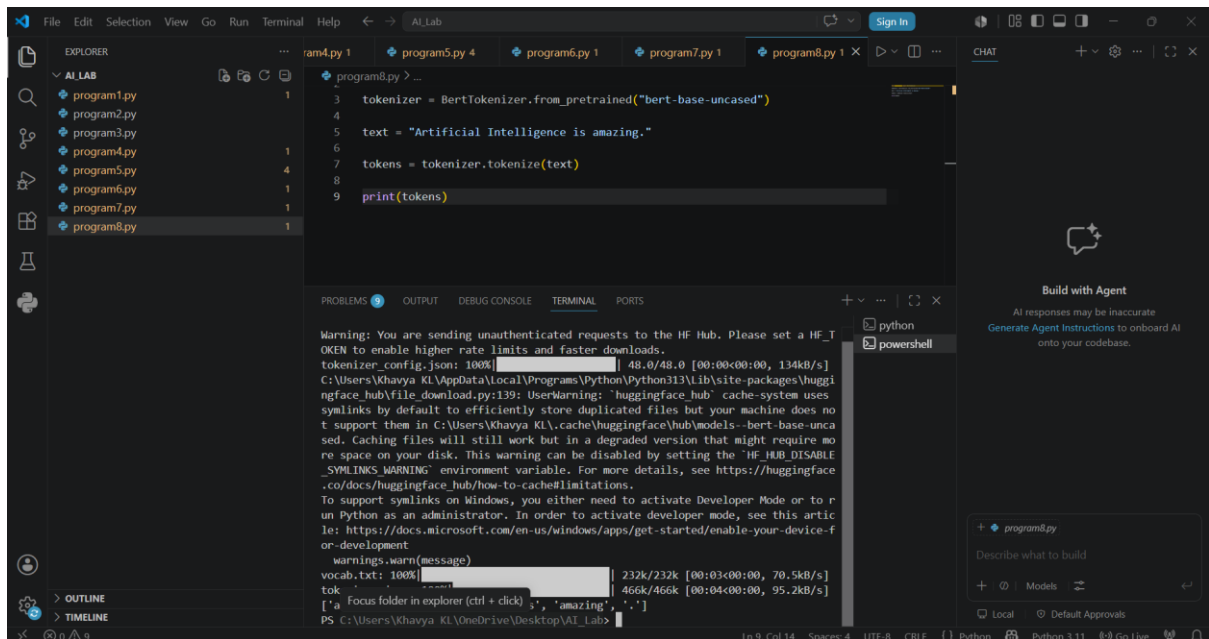
```
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program1.py
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program2.py
Scikit-learn Version: 1.9.0
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program3.py
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program4.py
  Name Grade
0  Asha    A
1  Ravi    B
2  Priya  A+
3  John   B+
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program5.py
NumPy: 2.3.5
Pandas: 2.3.3
Matplotlib: 3.10.8
Scikit-learn: 1.9.0
PyTorch: 2.13.0+cpu
Transformers: 5.14.1
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab>
```

EXP6

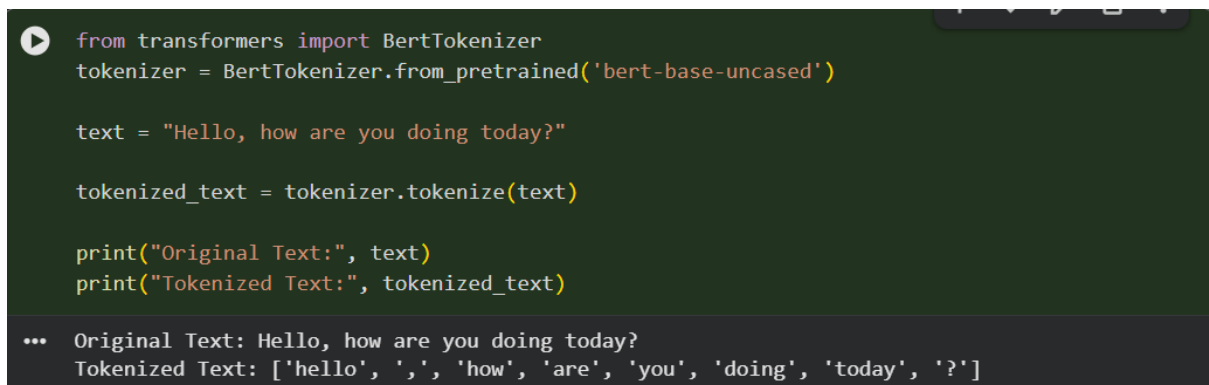
```
1 import torch
2
3 a = torch.tensor([1, 2, 3])
4
5 b = torch.tensor([4, 5, 6])
6
7 print("Tensor A:", a)
8 print("Tensor B:", b)
9
10 print("Addition:", a + b)
11 print("Subtraction:", a - b)
12 print("Multiplication:", a * b)
```

```
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program4.py
  Name Grade
0  Asha    A
1  Ravi    B
2  Priya  A+
3  John   B+
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program5.py
NumPy: 2.3.5
Pandas: 2.3.3
Matplotlib: 3.10.8
Scikit-learn: 1.9.0
PyTorch: 2.13.0+cpu
Transformers: 5.14.1
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab> python program6.py
Tensor A: tensor([1, 2, 3])
Tensor B: tensor([4, 5, 6])
Addition: tensor([5, 7, 9])
Subtraction: tensor([-3, -3, -3])
Multiplication: tensor([ 4, 10, 18])
PS C:\Users\Khavya KL\OneDrive\Desktop\AI_Lab>
```

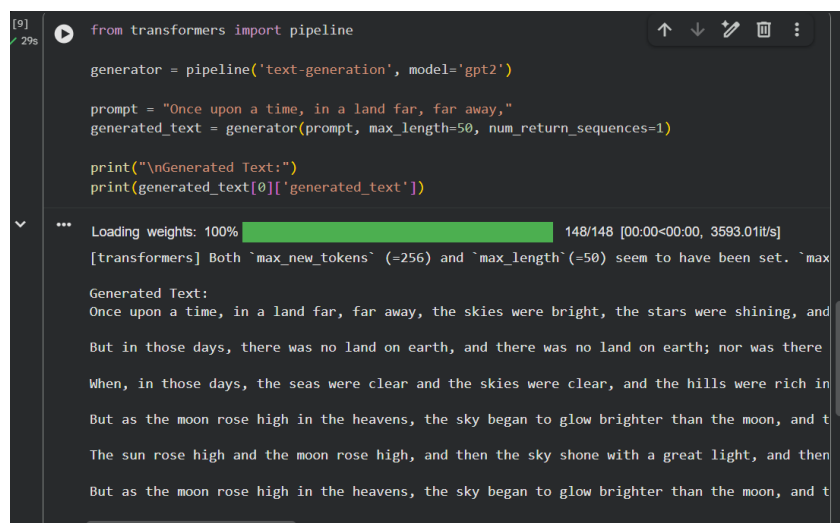
EXP7



EXP8



EXP9



EXP10

```
[11] from transformers import pipeline

sentiment_analyzer = pipeline('sentiment-analysis')

texts_to_analyze = [
    "I love using Hugging Face Transformers, it's amazing!",
    "This movie was absolutely terrible and boring.",
    "The weather today is neither good nor bad, just neutral."
]

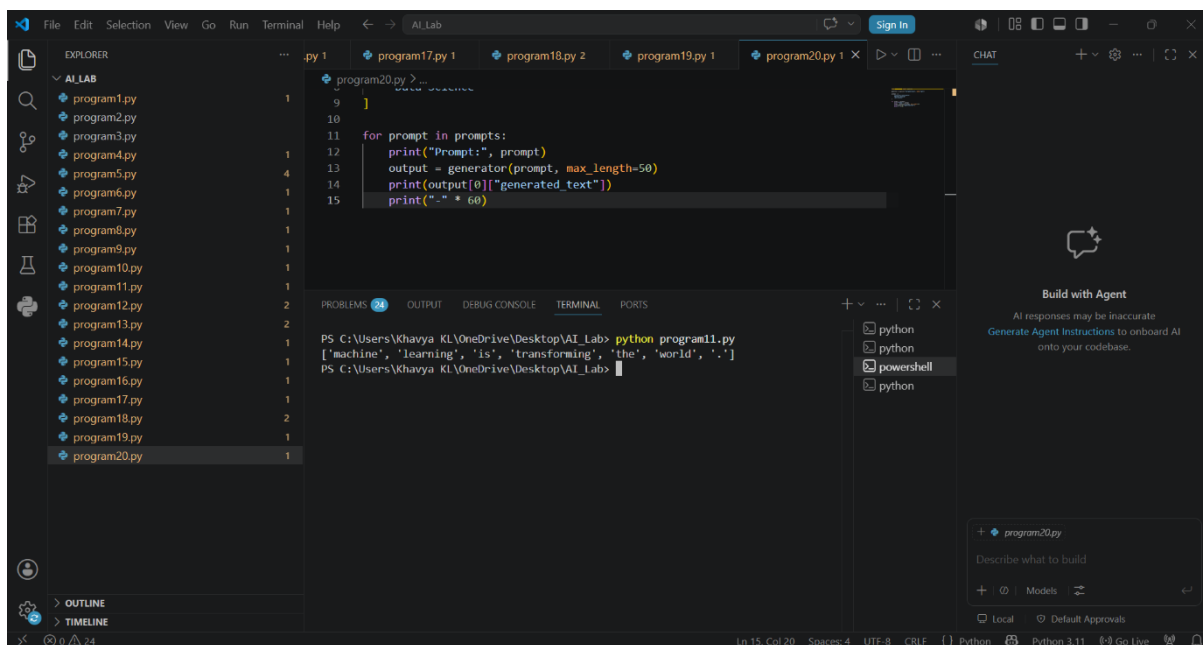
print("\nSentiment Analysis Results:")
for text in texts_to_analyze:
    result = sentiment_analyzer(text)
    print(f"Text: '{text}' => Sentiment: {result[0]['label']}, Score: {result[0]['score']:.4f}")
```

... [transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english. Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100% 104/104 [00:00<00:00, 3587.59it/s]

Sentiment Analysis Results:
Text: 'I love using Hugging Face Transformers, it's amazing!' => Sentiment: POSITIVE, Score: 0.9998
Text: 'This movie was absolutely terrible and boring.' => Sentiment: NEGATIVE, Score: 0.9998
Text: 'The weather today is neither good nor bad, just neutral.' => Sentiment: NEUTRAL, Score: 0.9998

EXP11



EXP12

```
[12] from transformers import BertModel, BertTokenizer
import torch

model_name = 'bert-base-uncased'
tokenizer_bert_embed = BertTokenizer.from_pretrained(model_name)
model_bert_embed = BertModel.from_pretrained(model_name)
text_for_embedding = "The quick brown fox jumps over the lazy dog."
inputs = tokenizer_bert_embed(text_for_embedding, return_tensors='pt', padding=True, truncat

with torch.no_grad():
    outputs = model_bert_embed(**inputs)
    last_hidden_states = outputs.last_hidden_state

print("\nText for Embedding:", text_for_embedding)
print("Token IDs:", inputs['input_ids'])
print("Shape of Last Hidden States (Embeddings):", last_hidden_states.shape)
print("Contextual Embedding for the word 'fox' (first token in the word 'fox'):")
tokens = tokenizer_bert_embed.convert_ids_to_tokens(inputs['input_ids'][0])

try:
    fox_index = tokens.index('fox')
    print(last_hidden_states[0, fox_index])
except ValueError:
    print("'fox' token not found or split differently. Displaying first token's embedding in
    print(last_hidden_states[0, 1])
```

```
Text for Embedding: The quick brown fox jumps over the lazy dog.
Token IDs: tensor([[ 101, 1996, 4248, 2829, 4419, 14523, 2058, 13971, 3899,
                    1012, 1021]])
Shape of Last Hidden States (Embeddings): torch.Size([1, 12, 768])
Contextual Embedding for the word 'fox' (first token in the word 'fox'):
```

tensor([[	3.2327e-02,	-2.3053e-01,	-1.7558e-01,	-1.1210e-01,	5.6922e-01,	2.3974e-01,	2.5757e-01,	7.2316e-01,	-4.9458e-02,	-3.9125e-01,	-6.3406e-02,	-3.9371e-01,	9.1828e-02,	-3.8244e-02,	-5.8939e-01,	-2.2635e-01,	1.2576e-01,	6.3714e-02,	-3.2564e-01,	-2.6846e-01,	-6.6565e-02,	-3.7944e-01,	-4.2205e-01,	-4.6839e-01,	4.6915e-01,	4.4356e-01,	3.0537e-01,	1.5674e-01,	-3.9826e-01,	5.7394e-01,	4.5419e-02,	-7.5581e-02,	6.4835e-01,	1.0582e-01,	-1.4846e-01,	2.0260e-01,	1.6430e-01,	-5.0165e-01,	-6.1023e-01,	-1.5120e-01,	-5.7296e-02,	-3.2515e-01,	-3.8738e-02,	4.2968e-01,	1.3564e-01,	-4.9326e-01,	-1.4896e-01,	2.3075e-02,	1.4931e+00,	-1.7319e-01,	-6.0812e-01,	6.9673e-01,	-7.4876e-01,	5.9866e-03,	-7.6011e-03,	1.9802e-01,	4.9726e-01,	-2.6701e-01,	-1.4594e-01,	5.6709e-02,	-4.2479e-01,	-1.0058e-01,	6.1553e-01,	3.9625e-01,	-1.1061e+00,	1.1707e+00,	-2.2555e-01,	3.1654e-01,	-5.7131e-01,	2.9942e-01,	1.6672e-01,	2.5195e-01,	-3.7495e-01,	4.0781e-01,	-2.6890e-01,	2.6460e-01,	3.0748e-01,	5.4302e-01,	7.1142e-02,	3.9932e-01,	1.7761e-01,	-1.4625e-01,	7.3488e-01,	6.2045e-01,	3.5595e-02,	8.7201e-02,	3.0367e-02,	-5.9669e-01,	-9.5982e-01,	9.9975e-01,	-4.0521e-01,	-5.6992e-01,	-7.4598e-01,	3.6058e-01,	-7.5506e-02,	-5.4216e-01,	3.0504e-01,	-4.7718e-01,	9.8175e-01,	-5.8771e-01,	-1.2619e-01,	3.4867e-02,	1.5839e-01,	2.4027e-01,	-8.7577e-01,	-4.8018e-01,	-6.6728e-01,	1.9521e-01,	-3.3373e-01,	2.9544e-01,	4.0223e-02,	3.2598e-02,	3.4094e-01,	7.8442e-02,	-1.3749e-02,	1.2430e+00,	-3.2380e-01,	-1.1047e-01,	-1.9270e-01,	2.0198e-02,	7.0248e-02,	9.1677e-03,	4.3610e-01,	4.2650e-01,	7.2925e-02,	-8.8347e-02,	6.9282e-02,	1.6670e-01,	-4.6056e-02,	-6.5476e-01,	2.2865e-01,	8.5058e-01,	9.7616e-02,	-6.2725e-01,	-2.4114e-01,	3.3457e-01,	-2.7981e-01,	-5.8425e-01,	-4.8522e-01,	-3.4071e-01,	6.1948e-01,	-5.5605e-01,	6.1118e-01,	2.8280e-01,	8.3666e-01,	4.9529e-01,	-3.0803e-01,	5.8963e-01,	6.9849e-01,	2.5557e-01,	-3.8148e-01,	7.1649e-02,	1.6248e-01,	-5.9162e-01,	8.1181e-02,	-1.2431e-01,	6.7709e-02,	2.7853e-01,	-1.2798e-01,	4.6879e-01,	1.7754e-01,	-3.2061e-01,	1.7833e-01,	1.4187e-01,	-1.8300e-01,	1.0846e-01,	5.4400e-01,	3.0176e-01,	-1.1292e+00,	1.0735e-01,	-1.1075e-01,	8.2842e-01,	7.0662e-01,	-2.4972e-01,	2.4601e-01,	3.3313e-02,	-9.2554e-02,	-6.1461e-02,	5.5533e-01,	-1.9118e-01,	-4.4987e-01,	5.3076e-01,	4.0544e-01,	3.6202e-01,	-4.0360e-01,	-5.9920e-01,	-2.0300e-01,	-4.8950e-03,	-1.1741e-01,	-7.2545e-01,	-2.4846e-01,	-3.9428e-01,	-1.9824e-01,	2.1585e-01,	8.0680e-01,	-7.4488e-01,	-3.0724e-01,	-9.9068e-01,	-1.6960e-01,	-3.6750e-01,	-4.4501e-01,	-8.1588e-01,	-1.4491e-01,	-1.0677e-01,	1.4398e-01,
-----------	-------------	--------------	--------------	--------------	-------------	-------------	-------------	-------------	--------------	--------------	--------------	--------------	-------------	--------------	--------------	--------------	-------------	-------------	--------------	--------------	--------------	--------------	--------------	--------------	-------------	-------------	-------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	--------------	--------------	--------------	--------------	--------------	--------------	-------------	-------------	--------------	--------------	-------------	-------------	--------------	--------------	-------------	--------------	-------------	--------------	-------------	-------------	--------------	--------------	-------------	--------------	--------------	-------------	-------------	--------------	-------------	--------------	-------------	--------------	-------------	-------------	-------------	--------------	-------------	--------------	-------------	-------------	-------------	-------------	-------------	-------------	--------------	-------------	-------------	-------------	-------------	-------------	--------------	--------------	-------------	--------------	--------------	--------------	-------------	--------------	--------------	-------------	--------------	-------------	--------------	--------------	-------------	-------------	-------------	--------------	--------------	--------------	-------------	--------------	-------------	-------------	-------------	-------------	-------------	--------------	-------------	--------------	--------------	--------------	-------------	-------------	-------------	-------------	-------------	-------------	--------------	-------------	-------------	--------------	--------------	-------------	-------------	-------------	--------------	--------------	-------------	--------------	--------------	--------------	--------------	-------------	--------------	-------------	-------------	-------------	-------------	--------------	-------------	-------------	-------------	--------------	-------------	-------------	--------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	-------------	--------------	-------------	--------------	-------------	-------------	--------------	-------------	-------------	--------------	--------------	-------------	--------------	--------------	-------------	-------------	-------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	-------------	-------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	-------------

EXP13

```
[13] 17s # New prompt for text generation
new_prompt = "The ancient prophecy foretold that a hero would rise when"

# Generate text with the new prompt
newly_generated_text = generator(new_prompt, max_length=100, num_return_sequences=1)

print("\nNewly Generated Text (GPT-2):")
print(newly_generated_text[0]['generated_text'])
```

... [transformers] Both `max\_new\_tokens` (=256) and `max\_length` (=100) seem to have been set. `ma

Newly Generated Text (GPT-2):
The ancient prophecy foretold that a hero would rise when the time was right, bring a mighty
Many of the heroes in our world, some centuries later, still carry the same legends. And thei
Let's take a look at the most famous of them:
THE BOSS (Yokozuna no Kyoukai).
It was the story of a young man in the city of Kurubara who was recruited by the army to fight
This story is very popular in Japan, but I wanted to give a different perspective.
In the manga, the hero has the power to use a sword.
The sword is made from the power of a holy spirit. The wielder has to use that power to do so

EXP14

```
[16] 4s with torch.no_grad():
    outputs = model(**inputs)

    answer_start_scores = outputs.start_logits
    answer_end_scores = outputs.end_logits

    answer_start = torch.argmax(answer_start_scores)
    answer_end = torch.argmax(answer_end_scores) + 1

    answer = tokenizer.decode(inputs["input_ids"][0, answer_start:answer_end])

    print(f"Question: {question}")
    print(f"Context: {context}")
    print("\nAnswer:")
    print(f"    Answer: {answer}")
```

... model.safetensors: reconstructing file: 100%  261MB / 261MB, 23.3MB/s
model.safetensors: downloading bytes:  246MB, 22.3MB/s
Loading weights: 100%  102/102 [00:00<00:00, 3129.94it/s]

Question: What is the capital of France?
Context: Paris is the capital and most populous city of France, with an estimated population
Answer:
 Answer: Paris

EXP15

```
import torch

new_sentence = "Transformers are powerful for many NLP tasks."

new_tokens = tokenizer_bert_embed.tokenize(new_sentence)
new_encoded_inputs = tokenizer_bert_embed(new_sentence, return_tensors='pt', padding=True, t

print("Original Sentence:", new_sentence)
print("Tokens:", new_tokens)
print("Input IDs:", new_encoded_inputs['input_ids'])
```

... Original Sentence: Transformers are powerful for many NLP tasks.
Tokens: ['transformers', 'are', 'powerful', 'for', 'many', 'nl', '##p', 'tasks', '.']
Input IDs: tensor([[101, 19081, 2024, 3928, 2005, 2116, 17953, 2361, 8518, 1012, 102]])

EXP16

```
list: bert_tokens_comparison
(10 items) ['the', 'quick', 'brown', 'fox', 'jumps', ...] lazy dog."

bert_tokens_comparison = tokenizer_bert_embed.tokenize(sentence_for_comparison)
bert_ids_comparison = tokenizer_bert_embed(sentence_for_comparison, return_tensors='pt')['input_ids']
gpt2_tokens_comparison = gpt2_tokenizer.tokenize(sentence_for_comparison)
gpt2_ids_comparison = gpt2_tokenizer(sentence_for_comparison, return_tensors='pt')['input_ids']
print("Sentence for Comparison:", sentence_for_comparison)
print("\n--- BERT Tokenization ---")
print("Tokens:", bert_tokens_comparison)
print("Input IDs:", bert_ids_comparison)

print("\n--- GPT-2 Tokenization ---")
print("Tokens:", gpt2_tokens_comparison)
print("Input IDs:", gpt2_ids_comparison)
```

... Sentence for Comparison: The quick brown fox jumps over the lazy dog.

--- BERT Tokenization ---
Tokens: ['the', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog', '.']
Input IDs: [101, 1996, 4248, 2829, 4419, 14523, 2058, 1996, 13971, 3899, 1012, 102]

--- GPT-2 Tokenization ---
Tokens: ['The', 'Ġquick', 'Ġbrown', 'Ġfox', 'Ġjumps', 'Ġover', 'Ġthe', 'Ġlazy', 'Ġdog', '.']
Input IDs: [464, 2068, 7586, 21831, 18045, 625, 262, 16931, 3290, 13]

EXP17

```
import torch
new_sentence_for_embedding = "The bank can be a river bank or a financial bank."

inputs_new = tokenizer_bert_embed(new_sentence_for_embedding, return_tensors='pt', padding=True)
with torch.no_grad():
    outputs_new = model_bert_embed(**inputs_new)
last_hidden_states_new = outputs_new.last_hidden_state

print("New Sentence for Embedding:", new_sentence_for_embedding)
print("Token IDs:", inputs_new['input_ids'])
print("Shape of Last Hidden States (Embeddings):", last_hidden_states_new.shape)
tokens_new = tokenizer_bert_embed.convert_ids_to_tokens(inputs_new['input_ids'][0])
bank_indices = [i for i, token in enumerate(tokens_new) if token == 'bank']
if len(bank_indices) >= 2:
    print(f"\nContextual Embedding for 'bank' (river bank):\n{last_hidden_states_new[0, bank_indices[0]]}")
    print(f"\nContextual Embedding for 'bank' (financial bank):\n{last_hidden_states_new[0, bank_indices[1]]}")
else:
    print("\nCould not find multiple occurrences of 'bank' to show different contextual embeddings")

...
-1.0123e-01,  3.1560e-01, -1.6739e-01,  6.2559e-01, -3.4738e-01,
 3.1464e-01,  4.0279e-01,  6.2402e-01,  2.0209e-02, -7.4008e-02,
-5.0567e-02, -8.2133e-02,  1.2748e-01,  1.5857e-01, -2.6074e-02,
 1.3872e-01,  5.2587e-02,  1.3719e-01,  1.9176e-02,  3.5060e-01,
 2.2116e-01,  1.8240e-01, -1.7959e-01, -6.3013e-01, -4.5395e-02,
 6.6231e-01,  1.8078e-01,  7.6074e-01, -1.7369e-01,  4.2544e-01,
-2.4063e-01, -2.1701e-01,  2.6190e-01, -3.5464e-01, -6.6863e-02,
 6.5975e-02,  4.0456e-02,  2.7010e-01,  5.0801e-01,  1.5016e-01
```

EXP18

```
[20] sentence2 = "A feline was resting on the rug."
print(f"Sentence 1: {sentence1}")
print(f"Sentence 2: {sentence2}")
inputs1 = tokenizer_bert_embed(sentence1, return_tensors='pt', padding=True, truncation=True)
with torch.no_grad():
    outputs1 = model_bert_embed(**inputs1)
embedding1 = outputs1.last_hidden_state[0, 0, :].numpy()
inputs2 = tokenizer_bert_embed(sentence2, return_tensors='pt', padding=True, truncation=True)
with torch.no_grad():
    outputs2 = model_bert_embed(**inputs2)
embedding2 = outputs2.last_hidden_state[0, 0, :].numpy()
embedding1_resaped = embedding1.reshape(1, -1)
embedding2_resaped = embedding2.reshape(1, -1)
similarity = cosine_similarity(embedding1_resaped, embedding2_resaped)[0][0]

print(f"\nEmbedding (CLS token) for Sentence 1 (first 5 values): {embedding1[:5]}")
print(f"Embedding (CLS token) for Sentence 2 (first 5 values): {embedding2[:5]}")
print(f"\nC cosine Similarity between the two sentence embeddings: {similarity:.4f}")

...
Sentence 1: The cat sat on the mat.
Sentence 2: A feline was resting on the rug.

Embedding (CLS token) for Sentence 1 (first 5 values): [-0.3642239 -0.05305341 -0.3673222 -0.37034437 -0.34506264]
Embedding (CLS token) for Sentence 2 (first 5 values): [-0.37034437 -0.34506264 -0.67988116 -0.36732222 -0.05305341]

Cosine Similarity between the two sentence embeddings: 0.9209
```

EXP19

```
[21] 26s new_gpt2_prompt = "In a futuristic city powered by renewable energy,"
generated_text_gpt2 = generator(new_gpt2_prompt, max_length=150, num_return_sequences=1)

print("\nGenerated Text (GPT-2) from new prompt:")
print(generated_text_gpt2[0]['generated_text'])
```

... [transformers] Both `max\_new\_tokens` (=256) and `max\_length` (=150) seem to have been set. `ma

Generated Text (GPT-2) from new prompt:
In a futuristic city powered by renewable energy, the future of cities will almost certainly
With a population of over 13 billion, the United States is the world's most populous country.
On average, the U.S. consumes 20% of its energy from renewable sources. So far, the U.S. has
The first step in the nation's clean energy revolution is making sure it's safe, efficient, a
This is one of many things the U.S. and other developed nations have come up with in their ef
Achieving clean energy requires a certain level of commitment, and some of it involves a lot
In the United States, clean energy is about 10% of the total energy consumption of the entire
In 2050, the U.S. will be the world's largest emitter of greenhouse gases.
At this point, we're going to have to make sure that our electricity sources are ready to use

EXP20

```
prompts_to_compare = [
    "The secret to a long and happy life is",
    "Artificial intelligence will revolutionize the way we",
    "The greatest challenge facing humanity today is"
]

print("Generating text for multiple prompts using GPT-2:\n")

for i, prompt in enumerate(prompts_to_compare):
    print(f"--- Prompt {i+1}: {prompt} ---")
    generated_response = generator(prompt, max_length=100, num_return_sequences=1, truncatio
    print(generated_response[0]['generated_text'])
    print("\n" + "-"*50 + "\n")
```

... [transformers] Both `max\_new\_tokens` (=256) and `max\_length` (=100) seem to have been set. `ma

Generating text for multiple prompts using GPT-2:

--- Prompt 1: The secret to a long and happy life is ---